

Context-Free Grammar for Procedural Stack Generation

Overview

We use a context-free grammar (CFG) to procedurally generate a city made up of coloured cube stacks on a flat grid. Each stack is a vertical column of cubes, where every cube is one of three colours: red, green, or blue. The grammar controls the structure of the city, and a set of random variables controls the specific choices made during generation.

Grammar Definition

Our CFG is defined as the 4-tuple $G = (V, \Sigma, R, S)$ where:

- $V = \{\text{CITY}, \text{STACK}, \text{COLUMN}, \text{CUBE}\}$ (non-terminals)
- $\Sigma = \{\text{R}, \text{G}, \text{B}\}$ (terminals)
- $S = \text{CITY}$ (start symbol)

The production rules R are:

$$\begin{aligned}\text{CITY} &\rightarrow \text{STACK CITY} \mid \text{STACK} \\ \text{STACK} &\rightarrow \text{COLUMN} \\ \text{COLUMN} &\rightarrow \text{CUBE COLUMN} \mid \text{CUBE} \\ \text{CUBE} &\rightarrow \text{R} \mid \text{G} \mid \text{B}\end{aligned}$$

What Each Rule Does

CITY is the top-level symbol. It expands into one or more stacks. The first production ($\text{CITY} \rightarrow \text{STACK CITY}$) adds a stack and keeps going. The second production ($\text{CITY} \rightarrow \text{STACK}$) adds one final stack and stops. This is how the grammar builds up a variable number of stacks.

STACK simply maps to a **COLUMN**. It exists as a separate non-terminal to keep the grammar readable—a stack *is* a column of cubes.

COLUMN works the same way as **CITY** but for cubes within a single stack. The recursive production adds a cube and continues; the base case adds one last cube and stops. This controls the height of each stack.

CUBE is where we reach a terminal symbol. Each cube becomes one of **R** (red), **G** (green), or **B** (blue).

Random Variables

The grammar defines the *structure*, but random variables decide the specific choices at each step.

RV	Distribution	Role in Grammar
Number of stacks	$\text{UniformInt}(n_{\min}, n_{\max})$	Decides how many times we apply $\text{CITY} \rightarrow \text{STACK CITY}$ before stopping with $\text{CITY} \rightarrow \text{STACK}$.
Stack height	$\text{UniformInt}(h_{\min}, h_{\max})$	Decides how many times we apply $\text{COLUMN} \rightarrow \text{CUBE COLUMN}$ before stopping with $\text{COLUMN} \rightarrow \text{CUBE}$.
Dominant colour	$\text{Uniform}(\{\text{R}, \text{G}, \text{B}\})$	Each stack gets one dominant colour chosen uniformly at random. This biases which terminal the CUBE rule produces.
Cube colour	Weighted choice	When expanding $\text{CUBE} \rightarrow \text{R} \mid \text{G} \mid \text{B}$, the dominant colour has probability p (default 0.7) and the other two split the remaining $1 - p$ equally.

For example, if a stack's dominant colour is B and $p = 0.7$, then each cube in that stack is blue with probability 0.7, red with probability 0.15, and green with probability 0.15.

Derivation Example

Here is a short derivation that produces a city with 3 stacks, where the first stack is 2 cubes tall with dominant colour B.

Step 1 — Expand CITY into stacks:

$$\begin{aligned} \text{CITY} &\Rightarrow \text{STACK CITY} \\ &\Rightarrow \text{STACK STACK CITY} \\ &\Rightarrow \text{STACK STACK STACK} \end{aligned}$$

Step 2 — Expand the first STACK into cubes:

$$\begin{aligned} \text{STACK} &\Rightarrow \text{COLUMN} \\ &\Rightarrow \text{CUBE COLUMN} \\ &\Rightarrow \text{CUBE CUBE} \end{aligned}$$

Step 3 — Expand each CUBE to a terminal colour:

$$\begin{aligned} \text{CUBE CUBE} &\Rightarrow \text{B CUBE} \\ &\Rightarrow \text{B B} \end{aligned}$$

The remaining two stacks are expanded the same way, each with their own randomly chosen height and dominant colour. The full city is the collection of all expanded stacks placed on a grid.

How It Connects to the Code

In the implementation, the grammar is stored as a Python dictionary where each non-terminal maps to its list of possible productions. The expansion functions walk through the grammar step by step:

- `expand_city()` applies the CITY rules to determine how many stacks to generate.
- `expand_column()` applies the COLUMN and CUBE rules to build each individual stack from bottom to top.
- The random variables plug in at each decision point, choosing which production to apply and which terminal colour to assign.

The result is a list of stacks, each with a grid position, height, dominant colour, and an ordered list of cube colours from bottom to top. This output is then passed to the rendering engine for visualization.